



数字信号处理器及其解决方案提供商

FDM320RV335 迁移指南

版本V1.0

青岛本原微电子有限公司

目 录

1. FDM320RV335 与 TMS320F28335 差异	- 1 -
1.1. 启动文件的制作	- 1 -
1.2. 代码安全模块 CSM	- 1 -
1.2.1. 修改密码区域后上锁	- 1 -
1.2.2. 上锁状态	- 1 -
1.3. 访存时的地址对齐	- 1 -
1.4. 高分辨率脉冲宽度调制器(HRPWM)模块	- 2 -
1.5. 不可屏蔽中断向量入口	- 2 -
2. 开发工具	- 3 -
2.1. 工程创建与结构	- 3 -
2.2. 编译配置	- 3 -
2.3. 调试配置	- 3 -
2.4. 工具相关	- 4 -
3. 编程模式差异	- 5 -
3.1. 内嵌汇编语句	- 5 -
3.2. 中断机制	- 6 -
3.3. C 语言特性	- 7 -
4. 芯片已知问题	- 8 -
4.1. 部分启动模式存在	- 8 -
4.2. JTAG 无法唤醒处理器的待机模式	- 8 -
4.3. 定时器模块	- 9 -
4.3.1. 定时器初始化	- 9 -

4.3.2. 定时器 TIMERxTIM 寄存器	- 9 -
4.4. 看门狗复位后连接仿真器异常	- 9 -
4.5. 事件捕捉模块 ECAP	- 9 -
4.5.1. 初始化 ECAP	- 9 -
4.5.2. APWM 模式	- 9 -
4.6. 正交编码器脉冲 EQEP 接口	- 10 -
4.6.1. EQEP 同步模式	- 10 -
4.6.2. EQEP 中断置位	- 10 -
4.7. 脉冲宽度调制 EPWM 模块	- 10 -
4.7.1. 时钟分频	- 10 -
4.7.2. 硬件同步问题	- 10 -
4.7.3. 首次波形异常	- 10 -
4.8. FREESOFT 不同模式会影响功能	- 10 -
5 工程迁移示范	- 11 -

1. FDM320RV335 与 TMS320F28335 差异

1.1. 启动文件的制作

TMS320F28335 采用 hex2000.exe 工具对工程生成的.out 进行 boot 文件格式的转换。

FDM320RV335 采用集成开发环境自带的 elf 转 bin 工具。

1.2. 代码安全模块 CSM

1.2.1. 修改密码区域后上锁

FDM320RV335 芯片在仿真器调试状态下，修改密码区域 0x67fff0~0x67fff 数值后，通过读密码区域/强制上锁/on-chip flash 配置等方式使芯片处于安全保护状态，此时仿真器不会断开连接。

竞品芯片在切换到安全保护状态时会断开仿真器连接。

此差异不影响 FDM320RV335 芯片的安全保护模式正常工作。

1.2.2. 上锁状态

FDM320RV335 芯片存储空间分为 PMEM 和 DMEM，分别用于存放代码和变量，其中 PMEM 是安全区域，受保护，DMEM 是非安全区。芯片上锁后，由于安全区域不被访问，导致无法访问存放代码的 PMEM，只能访问存放变量的 DMEM，这样无法进行代码调试。

TMS320F28335 存储区域分为 L0~L7,未区分 PMEM 和 DMEM，其中 L0~L3 是安全区域，L4~L7 是非安全区域。芯片上锁后，可以访问 L4~L7 区域继续进行代码调试。

1.3. 访存时的地址对齐

FDM320RV335 芯片总线访问要求地址长度和数据对齐，处理器内部最小的地址为字节地址（8bit），竞品芯片是字地址（16bit）。

进行总线访问时，要注意 FDM320RV335 与竞品芯片的偏移地址量差异。

1.4. 高分辨率脉冲宽度调制器(HRPWM)模块

微边缘定位器（micro edge positioner，MEP）的步长根据最坏情况下的工艺参数、操作温度和电压而变化。MEP 步长随电压的降低和温度的升高而增大，随电压的升高和温度的降低而减小。TMS320F28335 为高分辨率脉冲宽度调制器模块（High-Resolution Pulse Width Modulator, HRPWM）的应用程序提供了 MEP 缩放因子优化器（Scale Factor Optimizing Software, SFO），SFO 函数帮助在 HRPWM 运行时动态确定每个同步输出周期的 MEP 步数。

FDM320RV335 未提供该功能，建议可以预先在特定工作条件下，评估出需要的计数器值。

1.5. 不可屏蔽中断向量入口

FDM320RV335 芯片不可屏蔽中断的向量入口，需要在 trap.S 汇编文件中添加。如下图所示。

```
50
51
52 nmi_handler:
53     save
54     la a0,p_nmi_handler;
55     lw a0,0(a0)      ;
56     jalr a0;
57     restore
58     mret
59 trap_init:
60
61
62     la    t1, trap_handler
63     /* set mtvec mode 1 */
64     ori    t1, t1, 1
65     csrwr  mtvec, t1
66     /* enable nmie bit in mnstatus*/
67     csrsi  0x744, 0x8 /* TODO mnstatus reg name */
68     /* enable MIE in mstatus */
69     li t1, 0x7888
70     csrwr  mstatus, t1
71     /* csrsi  mstatus, 0x8 */
72     /* enable interrupt bits in mie */
73     li    t1, 0x888
74     csrrs  t1, mie, t1
75     /* set trap vector base address */
76     la    t1, nmi_handler
77     csrwr  0x806,t1
78     ret
79
80 external_interrupt_handler:
81
82     save
83     /* put claim address in s1 and IntVectTable in a1*/
```

图 1.1 不可屏蔽向量中断入口

2. 开发工具

TI 的 IDE（集成开发环境）CCS 与 RV335 的都基于 Eclipse 框架，在使用上大致相同，但是也存在部分差异，在此我们将一些开发过程中在软件使用上可能遇到的差别列举出来。

2.1. 工程创建与结构

TI 的 IDE 在创建工程时的向导与 RV335 的 IDE 在界面组织上略有不同，但是要填入的信息大致相同，主要都包括工程名称、目录，产品型号选择，工具链选择等，有一点区别是 CCS 需要填写仿真器的型号，而 RV335 默认使用 jlink 作为仿真器，所以不需要在这里指定仿真器型号。

创建工程后生成的工程模板和结构有所不同。Ti 会生成 .ccxml 文件，用于配置仿真器链接相关内容，RV335 没有这个文件，仿真器连接配置在 Debug Configuration 下，仿真器连接的检查在菜单栏的 Tools => Test Connection 中。Ti 使用的链接配置文件为 .cmd 文件，RV335 使用 C 标准链接文件格式的 .ld 文件。另外，创建 RV335 工程后，会自动生成一些外设的类库文件，方便用户使用。

2.2. 编译配置

CCS 工程编译相关配置在右键工程 => Properties => Build 下。展开后可以看到 Compiler, linker, Hex Utility 几个子配置项。而 RV335 的 IDE 的工程配置统一在右键工程 => Properties => C/C++ Build => Settings 下，点开 Settings 后在界面右侧可以对 Compiler, Assembler, Linker 做进一步配置。

2.3. 调试配置

通过 Debug Configuration 可以打开调试和运行配置的界面，TI 与 RV335 的配置界面存在一些差异。TI 使用 Main Tab 指定连接相关配置，RV335 的 IDE 中使用 Target Tab；TI 使用 Program Tab 指定要调试运行的工程，RV335 IDE 使用 Main Tab；TI 的 Flash 配置相关页面在 Target/ Flash Setting 下，RV335 IDE 直接将 On-Chip Flash 作为一个 Tab。

2.4. 工具相关

TI 使用 Hex2000 工具将调试用的可执行文件转换为 Hex 文件用于 boot; RV335 提供 elf2bin 工具将可执行文件转换为用于 boot 的 bin 文件. elf2bin 可以在 IDE 的安装目录下的 tools 文件夹中找到可执行程序, 也可以通过 IDE 工具栏的 Tools => Elf2Bin Tool 打开使用。

3. 编程模式差异

为了方便用户从 TI 向 RV335 平台的迁移，我们开发了 ti2rv 转换工具，同时，提供本文档，列出在开发过程中可能会遇到的两个平台编程和代码逻辑上的一些区别，方便用户参考从而更平滑地进行迁移工作。

3.1. 内嵌汇编语句

TI 在 C 代码中会通过定义宏与内嵌汇编结合，实现在 C 语言中快捷地插入常用的汇编指令来实现对底层硬件更好的控制，在使用习惯上 RV335 与 Ti 保持一致，但是由于 RV335 基于 RISC-V 架构，因此汇编指令与 TI 并不兼容，以下指令需要做出替换。

- TI 的 “ESTOP0” 指令替换为 “EBREAK”

```
// Ti
#define ESTOP0 asm("ESTOP0")

// RV335
#define ESTOP0 asm("EBREAK")
```

- TI 的 “EALLOW”, “EDIS”, “clrc DBGM”, “setc DBGM” 替换为 “NOP”

```
// Ti
#define ERTM asm("clrc DBGM")
#define DRTM asm("setc DBGM")
#define EALLOW asm("EALLOW")
#define EDIS asm("EDIS")

// RV335
#define ERTM asm("NOP")
#define DRTM asm("NOP")
#define EALLOW asm("NOP")
#define EDIS asm("NOP")
```

- TI 的 “IDLE” 替换为 “WFI”

```
// Ti
```

```
asm (" IDLE");  
  
// RV335  
  
asm (" WFI");
```

3.2. 中断机制

TI 使用 PIE 实现中断相关控制，而 RV335 使用 PLIC。在中断配置与使用上两者存在一些差别。

- TI 的中断处理函数需要声明 `__interrupt`，而 RV335 不需要。

```
// Ti  
  
__interrupt void adc_isr(void);  
  
// RV335  
  
void adc_isr(void);
```

- TI 与 RV335 对中断的配置与启动流程存在差异

```
// Ti  
  
DINT; // Disable CPU interrupt  
  
InitPieCtrl(); // initialize pie control registers  
  
IER = 0x0; // Disable CPU interrupt  
  
IFR = 0x0; // Clear cpu interrupt flags  
  
EALLOW;  
  
PieVectTable.EPwm1_INT = &epwm1_int_isr; // config isr  
  
EDIS;  
  
IER |= M_INT3; // Enable CPU INT  
  
PieCtrlRegs.PIECTRL.bit.ENPIE = 1; // Enable the PIE  
  
PieCtrlRegs.PIEIER3.bit.INTx1 = 1;  
  
EINT; // Enable Global interrupt INTM  
  
ERTM; // Enable Global realtime interrupt DBGM
```

```
// RV335

init_plic(); // init plic interrupt vector and priority

IntVectTable.st.EPWM1_INT = epwm1_isr; // config isr

trap_init(); // enable trap

enable_interrupt( GET_INT_NUM(IntVectTable.st.EPWM1_INT)); // enable interrupt
```

3.3. C 语言特性

- 为代码指定空间的方式有所区别，TI 使用 “# pragma CODE_SECTION” 进行空间映射，RV335 在函数声明时使用 “__attribute__” 关键字。

```
// Ti

#pragma CODE_SECTION(func, ".section")

void func (void) { // some code}

// RV335

void __attribute__((section(".section"))) func (void) { // some code}
```

- TI 使用自己平台特定的 cmd 文件作为连接脚本，RV335 使用 C 语言标准格式的连接脚本。

4. 芯片已知问题

4.1. 部分启动模式存在

FDM320RV335 启动方式分为 16 种模式，启动方式与竞品芯片的启动方式一致，如下表所示。

表 4-1 芯片启动差异

模式	GPIO87/XA15	GPIO86/XA14	GPIO85/XA13	GPIO84/XA12	说明
F	1	1	1	1	跳转到闪存
E	1	1	1	0	SCI-A boot
D	1	1	0	1	SPI-A 引导
C	1	1	0	0	I2C-A 引导
B	1	0	1	1	eCAN-A 引导
A	1	0	1	0	McBSP-A 引导
9	1	0	0	1	跳转到 XINTF x16
8	1	0	0	0	跳转到 XINTF x32
7	0	1	1	1	跳转到 OTP
6	0	1	1	0	并行 GPIO I/O 引导
5	0	1	0	1	并行 XINTF 引导
4	0	1	0	0	跳转至 SARAM
3	0	0	1	1	分支到检查引导模式
2	0	0	1	0	跳转到闪存,跳过 ADC 校准
1	0	0	0	1	跳转至 SARAM,跳过 ADC 校准
0	0	0	0	0	跳转至 SCI,跳过 ADC 校准

硅版本号为 A 的 FDM320RV335 芯片，设置为设置启动模式 0（跳转至 SCI，跳过 ADC 校准）时，使用仿真器进行 SCI 收发通信会存在问题。应避免使用此启动方式；其他 SPI、I2C、eCAN、McBSP 等启动模式不受此影响。

启动模式为 3 时，无法连接仿真器，应该避免使用此方式。

4.2. JTAG 无法唤醒处理器的待机模式

当 FDM320RV335 芯片处于待机模式时，无法用 Debug 模块进行唤醒。应避免使用 Debug 模块来唤醒待机模式。

4.3. 定时器模块

4.3.1. 定时器初始化

定时器初始化后第一次工作时，TIMERxTCR 寄存器中 TIF 位的置位时间会比设置期望值超出约十个定时器时钟周期。

需要用户评估该差异对系统的影响。

4.3.2. 定时器 TIMERxTIMH: TIMERxTIM 寄存器

定时器模块时钟配置为系统时钟 1 分频模式下，对定时器 TIMERxTIMH: TIMERxTIM 寄存器进行清零操作后，该 TIMERxTIMH: TIMERxTIM 寄存器会立刻重新加载 TIMERxPRDH: TIMERxPRD 寄存器中的值。

4.4. 看门狗复位后连接仿真器异常

FDM320RV335 芯片在仿真器调试状态下，看门狗模块触发系统复位后，仿真器会断开，IDE 中出现“hart reset”提示。

4.5. 事件捕捉模块 eCAP

4.5.1. 初始化 eCAP

FDM320RV335 芯片在初始化 eCAP 寄存器时，需要将 ECCTL2 寄存器的 TSCTRSTOP 位、ECCTL1 寄存器的 CAPLDEN 位的配置放在初始化程序最后的位置，以确保 CAP4-1 捕获寄存器中的数值正确。

4.5.2. APWM 模式

FDM320RV335 芯片 eCAP 模块配置为 APWM 模式时，设置脉冲周期值为 T、正脉冲宽度为 P，实际得到的输出波形脉冲周期值为 T+1、正脉冲宽度为 P+1。

当前版本的 FDM320RV335 芯片，eCAP 模块配置为 APWM 同步模式时，输出波形异常。设计中避免使用该模式。

4.6. 正交编码器脉冲 eQEP 接口

4.6.1. eQEP 同步模式

FDM320RV335 芯片中，eQEP 模块配置成同步输出模式后，S 管脚输出的同步信号的脉冲宽度比竞品芯片多 300ns。

4.6.2. eQEP 中断置位

FDM320RV335 芯片中，将 GPIO 复用为 eQEP 功能后，QFLG 寄存器的 SEL 会产生置位。使用时需要先将该位清零。

4.7. 脉冲宽度调制 ePWM 模块

4.7.1. 时钟分频

FDM320RV335 芯片中 ePWM 模块，在 Debug 模式下，软件断点位置可能影响 PWM 波形，全速运行 PWM 波形正常。

4.7.2. 硬件同步问题

FDM320RV335 芯片中 ePWM 模块，配置成硬件同步模式，ePWMSYNCI 输入有效时，同步相位不稳定，存在一个 TBCLK 误差。

4.7.3. 首次波形异常

FDM320RV335 芯片中 ePWM 模块，初始化配置后，第一个周期波形输出异常。

4.8. FREESoft 不同模式会影响功能

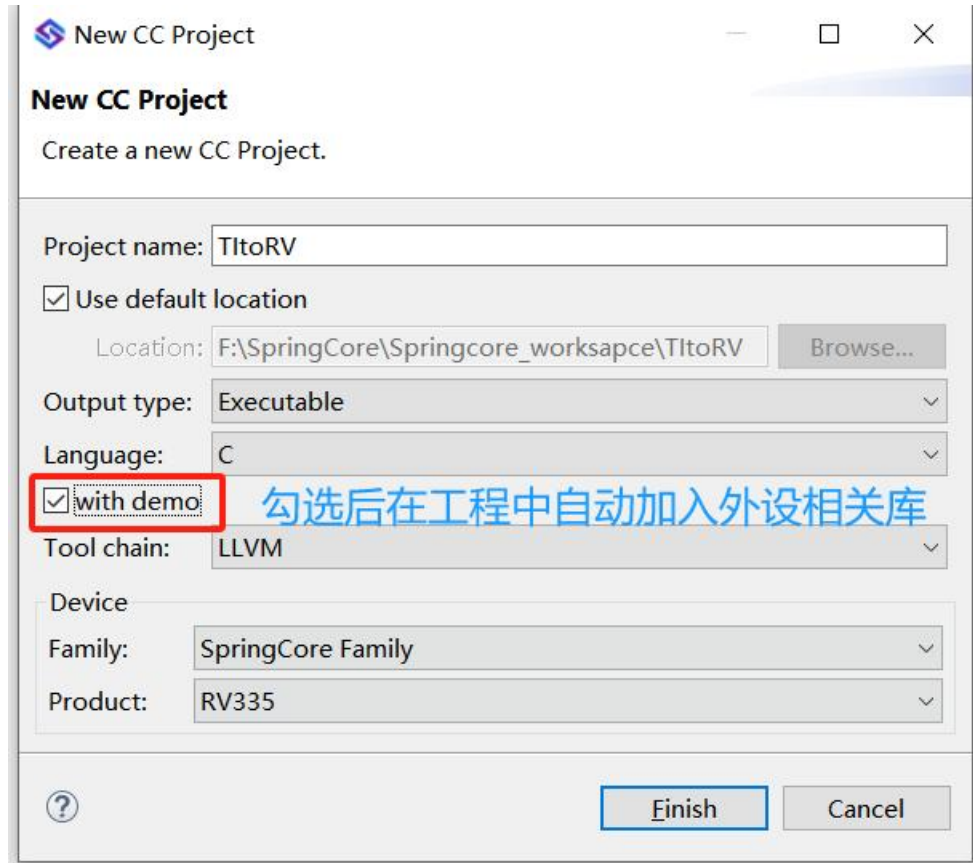
FDM320RV335 芯片中外设模块 FREESoft 位配置不同，在 Debug 模式下进行断点调试会影响功能。

如 McBSP 模块 FREESoft 位配置为 00，软件断点调试，功能测试不正常，需要配置为 10 模式，断点调试正常。

5 工程迁移示范

以下用一个 TI 的外设例程为例，展示用户迁移工程时的注意点。

1. 在 IDE 中创建新的 Rv335 项目



2. mian.c 文件如下

```
main.c
1 #include "RV335_Project.h" // Device Headerfile and Examples Include File
2 #include "RV335_SysCtrl.h"
3
4 int main(void)
5 {
6     // Initialize trap
7     trap_init();
8     init_priority();
9     // Initialize System Control
10    InitSysCtrl();
11    /* --- add your code below --- */
12
13
14    return 0;
15 }
16
```

2. 在保留上述 main 文件代码基础上，将 TI 工程的源码追加复制到新工程中

3. 删除无用库，并添加 RV335 外设库

```
//
// Included Files
//
#include "DSP28x_Project.h" // Device Headerfile and Examples Include File
//
```

删除

```
1 #include "RV335_Project.h" // Device Headerfile and Examples Include File
2 #include "RV335_SysCtrl.h"
3 #include "../routines/include/RV335_Device.h"
4 //
5 // ...
```

添加

4. 删除 TI 中断 PIE 相关配置,替换为 RV 中断 PLIC 配置

注册中断函数替换:

```
InitPieVectTable();

//
// Interrupts that are used in this example are re-mapped to
// ISR functions found within this file.
//
EALLOW; // This is needed to write to EALLOW protected registers
PieVectTable.SCIRXINTA = &rxaint_isr;
EDIS; // This is needed to disable write to EALLOW protected register
```

删除

```
IntVectTable.st.SCIRXINTA = &rxaint_isr;
```

添加

中断使能替换:

```
//
// Enable interrupts
//
PieCtrlRegs.PIEIER9.all = 0x0001; // Enable all SCIA RXINT interrupt
IER |= 0x0100; // enable PIEIER9, and INT9
EINT;

//

//
// Enable interrupts
//
enable_interrupt( GET_INT_NUM(IntVectTable.st.SCIRXINTA)); // Enable all SCIA RXINT interrupt
```

删除TI中断使能

替换为

删除文档中其他 PIE 的相关配置:

```
//
// Insert ISR Code here
//
PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
```

删除

5. 修改中断服务程序定义

```
7 //  
3 // rxaint_isr - ISR for PIE INT9.1 Connected to RXAINT SCI-A  
3 //  
3 _interrupt void  
1 rxaint_isr(void)  
2 {  
3     //  
4     // Insert ISR Code here  
5     //  
5     PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;  
-
```

RV不需要此声明，将该关键字删除

6. 编译，检查是否有不适配的 TI 指令插入